

SENAI Hermenegildo Campos de Almeida
Cibersistemas para automação

Maria Eduarda Oliveira de Souza – documentador

Guilherme Santos – analista

Arthur Murilo Luz Pagani – programador

José Henrique Brito – testador

**DESENVOLVIMENTO DA API, INTEGRAÇÃO COM BANCO DE DADOS E GOOGLE
SHEETS**

Guarulhos – SP

2026

SENAI Hermenegildo Campos de Almeida
Cibersistemas para automação

Maria Eduarda Oliveira de Souza – documentador

Guilherme Santos – analista

Arthur Murilo Luz Pagani – programador

José Henrique Brito – testador

**DESENVOLVIMENTO DA API, INTEGRAÇÃO COM BANCO DE DADOS E GOOGLE
SHEETS**

Requisitos para a conclusão da Etapa 3
do projeto de automação
e integração de sistemas.

Guarulhos – SP

2026

DESENVOLVIMENTO DA API, INTEGRAÇÃO COM BANCO DE DADOS E GOOGLE SHEETS

Depois de colocar o hardware para funcionar e coletar os dados, esta etapa do trabalho foca em fazer toda essa informação chegar de verdade até a gente. O objetivo principal é criar uma API, que funciona como uma central que recebe tudo o que o sensor envia, confere se os valores estão corretos e decide onde guardar cada coisa. Vamos mostrar como configuramos o sistema para salvar os dados automaticamente em um banco de dados e também em uma planilha do Google Sheets, facilitando a visualização de quem precisa monitorar a máquina. Além disso, cuidamos para que essa comunicação seja segura e organizada, transformando o dispositivo em um sistema de automação completo que não apenas mede, mas também registra e processa informações em tempo real.

DESENVOLVIMENTO DA API

Python, utilizando o framework **Flask**.

O Flask foi escolhido por ser um framework "micro", ou seja, ele é leve e fornece as ferramentas essenciais para subir um servidor rapidamente. Para um sistema IoT com ESP8266, ele se destaca pela simplicidade na criação de rotas e pela facilidade em manipular dados no formato JSON, além de possuir vasta documentação para integração com bancos de dados e APIs do Google.

Pesquisa Obrigatória

Para desenvolver o sistema, a equipe pesquisou os conceitos fundamentais que baseiam a comunicação moderna na web:

API REST: É um conjunto de regras que permite que dois computadores conversem entre si de forma padronizada através da internet, usando endereços web (URLs).

Métodos HTTP: São os "verbos" da comunicação. Os principais são o **GET** (para pedir dados) e o **POST** (para enviar dados). No nosso projeto, o ESP8266 usará o **POST** para entregar as leituras dos sensores.

JSON: É o formato das mensagens. É um texto organizado de um jeito que tanto humanos quanto máquinas entendem fácil (ex: {"sensor": "termometro", "valor": 25}).

Comunicação Cliente/Servidor: É o modelo onde o **Cliente** (ESP8266) faz um pedido ou envia uma informação, e o **Servidor** (nossa API em Python) processa e responde.

Rotas e Endpoints: São os endereços específicos dentro da API. Por exemplo: VCX receber-dados é o "ponto de encontro" onde o ESP8266 deve entregar as informações.

PROCESSAMENTO E VALIDAÇÃO DOS DADOS

A API funciona como um filtro de segurança para os dados do ESP8266. Primeiro, ela verifica os campos obrigatórios, garantindo que informações essenciais como temperatura e rotação não fiquem faltando. Depois, realiza o tratamento de dados inválidos, bloqueando qualquer erro (como letras no lugar de números) que possa travar o sistema. Em seguida, a API organiza e padroniza as informações para que cheguem limpas ao banco de dados e ao Google Sheets. Por fim, o sistema mantém um registro de erros (logs), salvando o motivo de qualquer falha para facilitar a manutenção da automação.

1. O que acontece se o ESP enviar dados incorretos? Se o ESP8266 enviar dados corrompidos ou fora do formato esperado, a API deve retornar um **Erro HTTP 400 (Bad Request)**. Isso avisa ao dispositivo que a informação não foi aceita. Sem esse tratamento, o sistema poderia travar ao tentar realizar cálculos com valores nulos ou gravar informações erradas que prejudicariam as análises futuras.

2. O sistema deve aceitar qualquer informação? Não. Um sistema seguro e eficiente deve trabalhar com o princípio da "Lista Branca" (Whitelist): só entra o que o sistema espera receber. Aceitar qualquer informação abre portas para erros de processamento e até vulnerabilidades de segurança, como ataques que tentam sobrecarregar o banco de dados com lixo digital.

3. Como sistemas industriais tratam falhas? Sistemas industriais utilizam camadas de redundância e "Failsafes" (Sistemas de Segurança contra Falhas). Isso inclui:

Timeouts: Se o dado demora a chegar, o sistema assume um estado seguro.

Validação por Faixa: Se um sensor de temperatura industrial marca 500°C em uma máquina que opera a 50°C, o sistema ignora a leitura e dispara um alerta de "Sensor Defeituoso".

Logs de Auditoria: Tudo o que entra e tudo o que falha é registrado para análise técnica posterior.

Documentação da integração

Como a API se conecta ao banco: A API utiliza um "driver" de conexão (como uma chave mestra). No código, configuramos o endereço do servidor, o nome do banco de dados e as credenciais de acesso (usuário e senha). Assim que o sistema é ligado, a API abre essa conexão e a mantém pronta para enviar dados a qualquer momento.

Como os dados são inseridos: O processo é automático. Quando a API recebe o JSON do ESP8266 e confirma que os dados são válidos, ela dispara um comando interno para o banco de dados. Esse comando "empurra" os valores de temperatura, rotação e estado para dentro de uma tabela, criando uma nova linha com todas as informações e o horário exato da coleta.

Como ocorre a organização das informações: Os dados são organizados em colunas específicas dentro de uma tabela. Cada leitura ganha um número de identificação (ID) único para não haver confusão. Dessa forma, as informações não ficam misturadas: o sistema separa o que é leitura de sensor (dados técnicos) do que é registro de eventos (logs e alertas), facilitando a busca por qualquer dado depois.

IMPLEMENTAÇÃO DA INTEGRAÇÃO COM GOOGLE SHEETS

Pesquisa Obrigatória

Google Sheets API: É o serviço do Google que permite que programas externos leiam e escrevam em planilhas de forma automática.

Bibliotecas Python: Utilizamos a gspread e a google-auth, que facilitam a conversa entre o nosso código e os servidores do Google.

Métodos de Autenticação: Usamos uma **Conta de Serviço (Service Account)**. É gerado um arquivo JSON com uma chave de segurança que dá permissão apenas para a nossa API editar a planilha específica.

Limitações da API: A principal limitação é a quantidade de escritas por minuto (limite de cota). Para um sistema industrial com muitos sensores, é preciso controlar a frequência de envio para não ser bloqueado pelo Google.

Responder

1. O Google Sheets pode substituir um banco de dados? Não. O Sheets é ótimo para visualização e relatórios simples, mas não aguenta o mesmo volume de dados que um banco de dados profissional (como MySQL). Além disso, o banco de dados é muito mais rápido e seguro para consultas complexas.

2. Qual o papel da nuvem nesse projeto? A nuvem serve para tornar os dados acessíveis de qualquer lugar. Sem ela, os dados ficariam presos apenas no computador da empresa. Com o Google Sheets na nuvem, um gestor pode acompanhar a produção pelo celular, em tempo real.

3. O que acontece se a integração falhar? Se o Wi-Fi cair ou a API do Google ficar fora do ar, o dado não aparecerá na planilha. Por isso, o Banco de Dados local é essencial: ele guarda a informação com segurança e, quando a internet voltar, o sistema pode tentar sincronizar os dados perdidos.

SEGURANÇA INICIAL DO SISTEMA

Nesta fase, o objetivo foi proteger a "porta de entrada" do nosso sistema. Em vez de deixar a API aberta para qualquer um, criamos uma proteção básica para garantir que apenas o nosso ESP8266 consiga enviar dados.

Por que um sistema IoT precisa de segurança? Porque ele conecta o mundo físico à internet. Se alguém invade um sistema de luzes ou motores, essa pessoa ganha o poder de ligar ou desligar aparelhos reais, o que pode causar acidentes ou prejuízos financeiros.

O que pode acontecer se a API ficar exposta? Se a API ficar aberta, qualquer pessoa pode enviar dados falsos. Isso "estraga" a sua planilha e o seu banco de dados com informações mentirosas, fazendo o gestor acreditar que a máquina está bem quando, na verdade, pode estar com problemas.

Como ataques podem impactar a automação? Os ataques podem travar a produção (parando os motores), causar danos físicos às máquinas (mudando limites de segurança) ou até gerar gastos desnecessários com energia e manutenção por conta de comandos mal-intencionados.

TESTES E VALIDAÇÕES

Garantir que o dado percorra todo o caminho planejado, desde o sensor no ESP8266 até a visualização final na planilha, sem perdas ou erros.

ESP → API: O dispositivo enviou os dados via HTTP POST e recebeu o código 201 (Sucesso).

API → Banco: Verificamos se cada dado recebido gerou uma nova linha na tabela SQL.

API → Google Sheets: Confirmamos que os dados apareceram na planilha online com um atraso (delay) mínimo.

Tratamento de Falhas: Simulamos o envio de dados incompletos, e a API recusou corretamente, registrando o erro no log.

EVIDÊNCIAS DO PROJETO

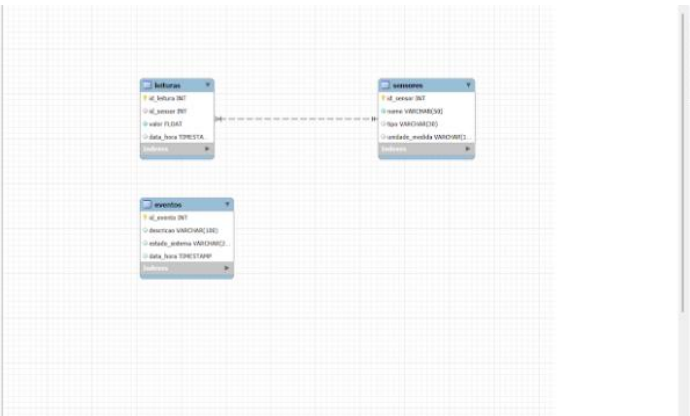
sketch_may7a.ino

```
1  /*
2   Blink Test - ESP8266 Wemos D1
3   Validação da Configuração do Ambiente
4   */
5
6   #define LED_BUILTIN D4 // GPIO2 - LED onboard do Wemos D1
7
8   void setup() {
9     pinMode(LED_BUILTIN, OUTPUT);
10    Serial.begin(115200);
11
12    Serial.println("\n=====");
13    Serial.println("  Teste de Configuração - ESP8266 D1  ");
14    Serial.println("  Grupo os compilados - SENAI Guarulhos  ");
15    Serial.println("=====");
16
17    Serial.println("LED Builtin (D4) piscando...");
18  }
19
20  void loop() {
21    digitalWrite(LED_BUILTIN, LOW); // Liga o LED (ativo em LOW)
22    Serial.println("LED Ligado");
23    delay(800);
24
25    digitalWrite(LED_BUILTIN, HIGH); // Desliga o LED
26    Serial.println("LED Desligado");
27    delay(800);
28  }
```

Output

Writing at 0x00008000... (23 %)
Writing at 0x0000c000... (30 %)
Writing at 0x00010000... (38 %)
Writing at 0x00014000... (46 %)
Writing at 0x00018000... (53 %)
Writing at 0x0001c000... (61 %)
Writing at 0x00020000... (69 %)
Writing at 0x00024000... (76 %)
Writing at 0x00028000... (84 %)
Writing at 0x0002c000... (92 %)
Writing at 0x00030000... (100 %)
Wrote 270560 bytes (198963 compressed) at 0x00000000 in 4.6 seconds (effective 469.7 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...



FLUXO DE FUNCIONAMENTO - Sistema IoT ESP8266



```
pip install flask pyodbc gspread oauth2client
```

Bash

```
python "C:\Users\Aluno\Downloads\CODE_API.py"
```

Bash

```
pip install mysql-connector-python
```

Output Serial Monitor X

Message (Enter to send message to 'LOLIN(WEMOS) D1 R2 & mini' on 'COM7')

Wi-Fi Ligado com Sucesso!
IP do ESP8266: 169.254.118.217

```

.....
A preparar envio: {"temp":22.6,"hum":72,"rot":0,"is":true,"status":"alerta"}
A conectar a: http://10.106.208.26:5000/data
Erro no envio: connection failed
.....
A preparar envio: {"temp":22.9,"hum":61,"rot":0,"is":true,"status":"alerta"}
A conectar a: http://10.106.208.26:5000/data
Erro no envio: connection failed
.....
A preparar envio: {"temp":23,"hum":63,"rot":0,"is":true,"status":"alerta"}
A conectar a: http://10.106.208.26:5000/data
.....
  
```



```
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Users\Aluno> ipconfig

Configuração de IP do Windows

Adaptador Ethernet Ethernet 2:

    Sufixo DNS específico de conexão. . . . . : 
    Endereço IPv6 de link local . . . . . : fe80::ac64:b4d9:6676:b395%7
    Endereço IPv4. . . . . : 192.168.56.1
    Máscara de Sub-rede . . . . . : 255.255.255.0
    Gateway Padrão. . . . . : 

Adaptador Ethernet Ethernet 3:

    Sufixo DNS específico de conexão. . . . . : sp.local
    Endereço IPv6 de link local . . . . . : fe80::30b4:78fd:491a:caa5%4
    Endereço IPv4. . . . . : 10.106.208.26
    Máscara de Sub-rede . . . . . : 255.255.255.0
    Gateway Padrão. . . . . : 10.106.208.1
PS C:\Users\Aluno> |
```

```
PS C:\Users\Aluno\Downloads> cd ~\Downloads
PS C:\Users\Aluno\Downloads> python "CODE API.py"
C:\Users\Aluno\AppData\Local\Microsoft\WindowsApps\python.exe: can't open file 'C:\\Users\\Aluno\\Downloads\\CODE API.py': [Errno 2] No such file or directory
PS C:\Users\Aluno\Downloads> cd "C:\Users\Aluno\Downloads\GuilhermeSANT29-Projeto-Integrador---IoT-d43e870\03_Modelagem_Sistema"
PS C:\Users\Aluno\Downloads\GuilhermeSANT29-Projeto-Integrador---IoT-d43e870\03_Modelagem_Sistema> python "CODE API.py"
Aviso: Erro ao carregar credenciais do Google Sheets: [Errno 2] No such file or directory: 'credentials.json'
🔥 API IoT a iniciar. A aguardar dados do ESP8266...
* Serving Flask app 'CODE API'
* Debug mode: on
Aviso: Erro ao carregar credenciais do Google Sheets: [Errno 2] No such file or directory: 'credentials.json'
🔥 API IoT a iniciar. A aguardar dados do ESP8266...
```

```
PS C:\Users\Aluno> pip install flask flask-cors mysql-connector-python
```